EECS 485 Final Exam
Fall 2025 SOLUTION

This is a 120 minute, closed-book exam.

You may use one cheat sheet, 8.5”x11”, double-sided.

No other notes or use of any computing or communication hardware (e.g., calculators, cell phones, smart glasses, etc.) is allowed.

For multiple-choice questions, indicate your response by filling in the appropriate bubble. Fill in the bubble completely, and make sure to erase completely if you change your answer. For free response questions, write your answers clearly in the space provided in each problem.

Collaboration or assistance is prohibited

- You are NOT allowed to collaborate with anyone else to answer these questions.
- You are NOT allowed to solicit assistance from anyone to answer these questions.

You are to abide by the University of Michigan/Engineering Honor Code. To receive a grade, please sign below to signify that you have kept the honor code pledge.

I have neither given nor received aid on this exam, nor have I concealed any violations of the Honor Code.

Signature: _____

Name: _____

Uniqname: _____

UMID: _____

Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

Make sure to write your uniqname at the top of every page of the exam.

This page is intentionally left blank

Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

Multiple Choice (40 points)

Demo Question: How should I fill in my responses for a Multiple Choice question?

- **A. Fill in the chosen bubble completely and only select ONE per question!**

1. A client is streaming data from a server over TCP, and suddenly packets are dropped due to network congestion. Which outcome is most consistent with how TCP behaves?

◯ A. Some data may be permanently lost
◯ B. The number of ACK packets will increase compared to the number of data packets
◯ C. The number of ACK packets will stay the same as before the congestion
◯ D. The sender's congestion window will grow
◯ E. The sender's congestion window will shrink

Solution: E. When congestion increases, TCP's congestion control algorithm reacts by shrinking the congestion window, which reduces the sender's sending rate. The connection still stays alive. Choices A, C, and D are wrong since TCP will still guarantee reliable, in-order delivery (no loss, no skipping, no out-of-order delivery) to the application. TCP also never switches protocols, making Choice B wrong as well.

2. An online multiplayer game experiences a problem where player positions appear to "jump" on screen rather than moving smoothly. Investigation reveals that when packet 50 is lost, packets 51-70 arrive but aren't processed until packet 50 is retransmitted. Which networking concept is the most responsible for this behavior?

◯ A. UDP
◯ B. Congestion control
◯ C. IP
◯ D. TCP
◯ E. Flow Control

Solution: D, TCP Enforces ordered delivery so if packet 50 is missing, then later packets are buffered and withheld until 50 is retransmitted.

Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

3. When using MinHash and shingling to detect near-duplicates, how does increasing the shingle size k (e.g., moving from 2-shingles to 10-shingles) affect similarity detection?

◯ A. More documents appear similar
◯ B. Fewer documents appear similar
◯ C. Only affects the speed of computation.
◯ D. Increases the probability of hash collisions in MinHash
◯ E. Allows the detection of duplicates even if the words are in a different order.

Solution:B. Increasing (k) makes each shingle a longer exact phrase, so small edits affect many shingles and the similarity drops. In other words, a larger shingle size means documents must contain larger sections that match exactly.

4. You are calculating the TF-IDF weight for the term "kangaroo." In which scenario does this term receive the highest weight in a specific document D?

◯ A. "Kangaroo" appears once in D and appears in all other documents in the collection.
◯ B. "Kangaroo" appears frequently in D and appears in all other documents in the collection.
◯ C. "Kangaroo" appears once in D and is very rare in the rest of the collection.
◯ D. "Kangaroo" appears frequently in D and is very rare in the rest of the collection.
◯ E. "Kangaroo" does not appear in D

Solution:D. TF-IDF is biggest when the term shows up many times in D (high TF) but is rare across the whole collection (high IDF).

5. Suppose a new website links to every other website but receives no incoming links. What will most likely happen after recomputing PageRank to convergence?

◯ A. Every other page gets a large PageRank boost
◯ B. PageRank values of existing pages barely change
◯ C. The new page accumulates high PageRank
◯ D. Pages with many incoming links will gain significantly more PageRank than pages with few
◯ E. The dampening factor will change to compensate

Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

Solution: B. PageRank only increases when other pages link to the page. H has no incoming links, so it has almost no PageRank to give out. Even though it links to every page, it's spreading a tiny amount of rank across many outgoing edges, so nothing really changes.

6. Why might a programmer choose processes over threads?
   - ◯ A. Processes execute faster than threads
   - ◯ B. Processes can share data more easily
   - ◯ C. Processes provide isolation to limit damage from buggy or untrusted code
   - ◯ D. Processes work better for I/O-bound workloads
   - ◯ E. Processes don't require synchronization primitives for shared data structures

Solution: C. Processes have separate address spaces, so bugs or crashes are contained; threads share memory and can corrupt the whole process without careful synchronization.

7. A user visits example.com. Later, the IP address changes, but their next visit still goes to the old server. Why?
   - ◯ A. The user's browser always stores DNS permanently
   - ◯ B. DNS resolvers cache TTL records
   - ◯ C. The iterative DNS resolution algorithm stops after a cache hit for an NS record
   - ◯ D. DNS caches only root nameserver data
   - ◯ E. DNS resolvers cache A records until their TTL expires

Solution: E. DNS caching respects TTL, so results persist until expiration.

8. Which is a key distinction between HITS and PageRank?
   - ◯ A. PageRank computes two scores (hub & authority), HITS computes one
   - ◯ B. HITS is query-dependent; PageRank is query-independent
   - ◯ C. PageRank requires no normalization; HITS does not normalize
   - ◯ D. HITS uses a damping factor; PageRank does not
   - ◯ E. PageRank looks only at outgoing edges; HITS looks only at incoming edges

Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

Solution: B. HITS generates a root set per query; PageRank runs once and is query-independent.

9. Which Python function is used to prevent busy-waiting?

◯ A. threading.Thread.join()
◯ B. socket.socket.listen()
◯ C. socket.socket.settimeout()
◯ D. socket.socket.setsockopt()
◯ E. time.sleep()

Solution: E. time.sleep() is the only function of these that can be used in a loop to yield control to the operating system to switch tasks.

10. Which of the following about the MapReduce framework in Project 4 is FALSE?

◯ A. The manager and workers can be on different computers, so long as they share a file system
◯ B. Reducer executables can be written in C++
◯ C. The contents of each input file to the reducers must be in sorted order
◯ D. The pipeline produces the same number of output files as the number of mappers
◯ E. If any worker is performing mapping, no other worker can be performing reducing

Solution: D. The pipeline produces the same number of output files as the number of **reducers**, not mappers. C is true, because the **mappers** are responsible for sorting all their output files, which become the input files to the reducers. See the P4 spec section at https://eecs485staff.github.io/p4-mapreduce/#mapping-worker.

11. You have an inverted index with 1,000 terms and 20,000 documents. You are segmenting this index between 10 servers. Which of the following are TRUE?

◯ A. When segmenting by term, each server will be responsible for 1,000 terms
◯ B. When segmenting by term, each server will be responsible for 2,000 documents
◯ C. When segmenting by document, each server will be responsible for 100 terms

Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

◯ D. When segmenting by document, each server will be responsible for 1,000 documents

◯ E. When segmenting by document, each server will be responsible for 1,000 terms

Solution: E. When segmenting by document, each server is responsible for every term, but not every document.

12. A company uses round-robin DNS to distribute requests across three servers. Suddenly, one server becomes overloaded while the others remain idle. Which explanation best accounts for this imbalance?
    - ◯ A. Round-robin DNS changes the HTML structure of responses
    - ◯ B. The DNS system is randomly dropping packets
    - ◯ C. Round-robin DNS does not consider per-request computational load
    - ◯ D. Threads inside the busy server are blocking DNS rotation
    - ◯ E. DNS always sends 90% of traffic to the first IP in its rotation

Solution: C. DNS has no way of knowing what a server's load is.

13. Which scenario best justifies database sharding rather than replication?
    - ◯ A. Data fits easily on a single machine
    - ◯ B. Most queries require consistent reads
    - ◯ C. Reads vastly outnumber writes
    - ◯ D. The dataset is too large to store multiple complete copies
    - ◯ E. The network frequently partitions

Solution: D - When the dataset is too large to store full copies, you're forced to partition it across machines, which is exactly what sharding provides. Replication doesn't solve storage limits, it only increases redundancy and read throughput.

14. A distributed database is designed so that if a partition occurs, the system returns errors rather than stale data. This design choice implies:
    - ◯ A. The system is sharded rather than replicated
    - ◯ B. The system has both consistency and availability
    - ◯ C. The system is violating the CAP theorem
    - ◯ D. Availability is chosen over consistency
    - ◯ E. Consistency is chosen over availability

Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

Solution: E - It is possible that during a partition (two nodes on the network unable to communicate) certain nodes are updated while others aren't. Choosing to return an error instead of potentially stale data enforces consistency.

15. A recommender system uses a nearest-neighbor algorithm with cosine similarity. The developer does not normalize the vectors. What will the difference be compared to an implementation that did normalize the vectors?

◯ A. Users who rate more items will be selected as a nearest neighbor more often
◯ B. Users who rate fewer items will be selected as a nearest neighbor more often
◯ C. Items rated fewer times will be recommended more often
◯ D. Items with more ratings will be recommended more often
◯ E. There will be no difference

Solution: E. The operations needed to compute cosine similarity can be simplified by normalizing the vectors, but the result will be the same whether or not they were normalized. See DeOrio-Live lecture 15 slides 36-37, Almomani lecture 15 slides 43-44, Kloosterman lecture 15 slide 44.

16. Why would a recommender system use inverse user frequency in its computations?

◯ A. Users that rate few items give more valuable information
◯ B. Users prefer recommendations that are given to few other users
◯ C. It prevents extreme content from being recommended to too many users
◯ D. It is a fundamental part of content-based filtering
◯ E. It helps recommend items that only a small community of users has rated

Solution: E - Inverse user frequency down-weights items that many have rated and up-weights items that a small group have rated.

17. In a sealed-bid, second-price auction, you value an item at $100. Your research suggests other bidders value it between $60-$120. A consultant advises you to bid $85 to "split the difference and maximize expected profit." Why is this advice flawed?

◯ A. You should bid higher ($120) to ensure you win against the strongest competitor
◯ B. The advice ignores that your bid doesn't affect whether you win, only what you pay

Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

◯ C. Bidding below your true value could cause you to lose to someone who values the item less than you

◯ D. You should randomize your bid to prevent strategic behavior

◯ E. The correct strategy is to bid $0.01 below your true value

Solution: C - In a sealed-bid second-price auction, the optimal strategy is to bid the item's value.

18. Which part of the Bitcoin system uses public-key cryptography?

◯ A. Linking blocks to previous blocks in the chain

◯ B. Verifying transfers were authorized by the sender

◯ C. Verifying the same coins were not spent twice

◯ D. Hashing blocks to determine which miner can add a block

◯ E. Encrypting private wallets' transaction history

Solution: B - Senders sign transactions with their private key to show they authorize the transaction. A does not use cryptography. C uses the longest-chain consensus. D uses a cryptographic hash (sha256), but hashes are not public-key cryptography. There is no such thing as a private wallet for option E.

19. The Bitcoin protocol requires miners to perform Proof-of-Work computations. What is the primary security purpose of this mechanism?

◯ A. It encrypts the transaction history to make it impossible to change

◯ B. It increases the transaction throughput of the network by parallelizing the verification process across millions of GPUs

◯ C. It validates the identity of the sender more times

◯ D. It makes it prohibitively expensive to create blocks containing double spends

◯ E. It makes it prohibitively expensive to forge an alternate history of the blockchain

Solution: E - To forge an alternate history by adding many blocks, proof-of-work requires an attacker to control a large portion of all the mining power in the system.

Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

20. An authoritarian government with control of a country's Internet wants to block Tor by identifying and blocking all Tor entry nodes. However, even after blocking thousands of known entry nodes, citizens continue accessing Tor. What property of Tor's design makes this approach difficult?

◯ A. The government needs to block exit nodes instead

◯ B. Tor nodes are constantly being added and removed

◯ C. Any Tor node can act as an exit node

◯ D. Blocking Tor nodes would also block legitimate VPN services, causing economic damage

◯ E. Encryption prevents VPN traffic from being detected

Solution: B - Tor nodes are dynamically added and removed. A and C do not reflect the structure of the Tor network. D is not true because the IPs of Tor entry nodes are published, allowing other VPNs to not be blocked. E is not true because the destination IP of packets must be unencrypted even when the payload is.

Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

Free Response 1: Pie-ority Manager 485 (30 points)

The Cats of the Grand Pie Consortium have hired you to manage their distributed pie-baking system. Each pie-baking request is a job with priority **HIGH** or **LOW**. HIGH-priority pies must always be baked before LOW-priority pies.

You will implement a Manager that assigns jobs to Workers while respecting **Priority Rules**:

- If no Workers are available to process a new HIGH-priority job, the Manager must **cancel** a running LOW-priority job so that the HIGH-priority job can be processed.
- Cancelled jobs must be restarted later—**unless** the Worker finishes the job before the cancel request arrives. When this occurs, the manager may still receive a `job_completed` message for a job it intended to cancel; in that case, treat it as completed and **do not restart it**.
- You must receive confirmation that a canceled job has been cancelled or completed before launching it again. A worker will ever only send one of `job_completed` and `job_cancelled` for a given job.
- A worker cannot be assigned a new job until it has messaged the manager that the current job was **completed** or **cancelled**.
- You may not cancel more jobs than the number of HIGH-priority jobs, and you must not cancel a job if there are enough idle workers to run the incoming jobs.

Threading Requirements

Your Manager must use **two threads total** (including the main thread):

TCP Listening Thread:

- Runs the provided `tcp_server` function, forwarding received messages to `handle_tcp`.
- Updates internal state for the main thread to act upon.

Main Thread:

- Start the TCP listening thread.
- Continuously assign jobs to Workers as per the *Priority Rules*.

Avoid busy waiting wherever possible.

Simplifying Assumptions:

- Jobs of the same priority may be executed in any order.
- Workers do not die and will always eventually respond to all messages. All messages will be delivered.
- All messages received are well formatted.

Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

Provided TCP helper functions

You are encouraged to use these functions in your implementation. Note that these **are not** class members

```
def tcp_client(host, port, msg):
"""
Creates a socket connection to (host, port) and sends the msg dictionary.
"""

def tcp_server(host, port, handler, signals):
"""
Listens for TCP connections on (host, port) and receives messages until
signals["shutdown"] is set to True. When a message is received,
handler(msg) is called, where msg is a dictionary.
"""
```

Managers can **receive** the following messages via **TCP**:

From a client:
{
"type": "launch_job",
"priority": “HIGH” | “LOW”,
"command": <string>
}
	From worker:
	{
	"type": "register",
	"host": <string>,
	"port": <int>
	}

From a worker:
{
"type": "job_completed",
"job_id": <int>
}
	From a worker:
	{
	"type": "job_cancelled",
	"job_id": <int>
	}

Managers can **send** the following messages via **TCP:**

To a worker:
{
"type": "start_job",
"job_id": <int>,
"command": <string>
}
	To a worker:
	{
	"type": "cancel_job",
	"job_id": <int>
	}

Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

Provided Member Functions

You are encouraged to use these functions in your implementation. You may assume these methods **function correctly as specified**, regardless of your implementation details. Note that these **are** class members. You do not need to add member variables that support these functions.

```
def assign_job_to_worker(self, host, port, job_id, command: str):
"""
Assign a job to a worker and send the start_job message.
"""

def find_running_low_job(self):
"""
Returns (worker host, worker port, job_id) for a LOW priority job currently
assigned to a worker, or (None, None, None) if none found. A job is still
running even if you have sent a cancel message to its worker.
"""
```

```
class Manager:
   def __init__(self, host: str, port: int):
      self.signals = {"shutdown": False}
      self.idle_workers = set() # Set of (host, port) tuples
      self.next_job_id = 0
      self.low_queue = [] # list of job_ids of low priority jobs
      # Add additional data structures here, if necessary
      self.high_queue = []                # List of job_ids
      self.cancelling = set()             # Set of job_ids
      self.job_commands = {}              # job_id -> command
      tcp_thread = threading.Thread(
            target=tcp_server,
            args=(host, port, self.handle_tcp, self.signals)
      )
      tcp_thread.start()
      self.assign_jobs()  # Write implementation below
      tcp_thread.join()
```

Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

```
def assign_jobs(self):
    while not self.signals["shutdown"]:
        # if we have HIGH jobs waiting, but no idle workers, cancel a low
        # priority job and assign it to the worker.
        if self.high_queue and not self.idle_workers and \
            len(self.cancelling) < len(self.high_queue):

            # We are assuming that self.find_running_low_job()
            #  does not return the same job_id every time. Student solutions
            #  were allowed to make that same assumption.
            host, port, low_job_id = self.find_running_low_job()

            if host and (low_job_id not in self.cancelling):
                self.cancelling.add(low_job_id)
                tcp_client(
                    host,
                    port,
                    {"type": "cancel_job", "job_id": low_job_id},
                )
            # Note this solution only cancels one job at a time, but
            #  can cancel additional jobs next time around the while loop.

        if self.idle_workers and (self.high_queue or self.low_queue):
            if self.high_queue:
                job_id = self.high_queue.pop(0)
            elif self.low_queue:
                job_id = self.low_queue.pop(0)

            worker = self.idle_workers.pop()
            command = self.job_commands[job_id]
            self.assign_job_to_worker(worker[0], worker[1], job_id, command)

        time.sleep(0.1)
```

Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

```
def handle_tcp(self, message):
   # register a new worker as idle
   if message["type"] == "register":
       worker = (message["host"], message["port"])
       self.idle_workers.add(worker)

   elif message["type"] == "shutdown":
         self.signals["shutdown"] = True

   elif message["type"] == "launch_job":
       # create new job and add to appropriate queue
       job_id = self.next_id
       self.next_job_id += 1
       self.job_commands[job_id] = message["command"]

       if message["priority"] == "HIGH":
           self.high_queue.append(job_id)
       else:
           self.low_queue.append(job_id)
```

Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

```
elif message["type"] == "job_completed":
    job_id = message["job_id"]
    # Add worker back to idle pool. You can assume this member function
    # is implemented for you and works correctly.
    worker = self.find_worker_by_job_id(job_id)
    self.idle_workers.add(worker)
    print(f"Job {job_id} complete!")
```

```
# job completed before cancel msg was received by worker
# (don’t restart)
if job_id in self.cancelling:
    self.cancelling.remove(job_id)

# clean up the job since it's done
del self.job_commands[job_id]
```

```
elif message["type"] == "job_cancelled":
    job_id = message["job_id"]
    # Add worker back to idle pool. You can assume this member function
    # is implemented for you and works correctly.
    worker = self.find_worker_by_job_id(job_id)
    self.idle_workers.add(worker)
```

```
# successfully cancelled, remove from cancelling set
self.cancelling.remove(job_id)

# put job back in queue (restart it)
# HIGH jobs can't be cancelled, so the job must be LOW priority
self.low_queue.append(job_id)
```

Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

Free Response 2: Speedy Speedy Robots (30 points)

eecs485staff has partnered with a local warehouse robot provider to analyze robot movement efficiency! Write a **two-stage MapReduce pipeline** that identifies the average transition time between warehouse zones and determines which robots are moving the slowest for each transition path.

Input

map1.py will receive warehouse log file entries in the following format:
`<robot_id> <timestamp> <zone> <event>`

Example input:

```
r2 700.0 produce load
r2 800.0 dairy unload
r1 100.0 bakery load
r2 100.0 freezer load
r1 200.0 bakery unload
r1 900.0 produce load
r1 1005.0 dairy unload
r2 600.0 bakery unload
```

`robot_id:` unique robot identifier represented as the prefix "r" with a number
`timestamp:` timestamp (in minutes since start)
`zone:` the warehouse zone the event is happening in
`event:` "load" or "unload"

Stage 1

The first stage must identify **all zone transitions for each robot** and calculate the **duration of each transition**. A zone transition occurs when a robot unloads in a different zone than where it loaded. It also occurs when a robot moves from a zone it unloaded into a different zone to load. Loading and unloading in the same zone is **not** a zone transition, nor is loading in the same zone that the robot was unloaded in. The transition duration is calculated as the difference between the timestamp when the robot arrives at the new zone and the timestamp when it was last seen in the previous zone.

You may assume loading always happens prior in time to unloading for a given robot, and that every unload is preceded in time by exactly one load. However, the input file may not be sorted by the timestamp.

Stage 2

The second stage must calculate the **average duration** for each unique transition across all robots as well as determine the **slowest robot** - the robot with the longest time for that path. When the same robot undergoes the same zone transition multiple times, **each time counts**

Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

separately when calculating averages and determining the robot with the slowest transition. When determining the slowest robot, you may break ties however you wish.

Output

For each unique zone transition path, display the transition statistics in the following format based on how many robots have taken the zone transition path. If only one robot has taken the zone transition path, the <avg_duration> and <slowest_duration> fields will be identical.

<transition_path> <avg_duration> <slowest_robot> <slowest_duration>

Example Output (from the input mentioned above):

```
bakery->produce 400.0 r1 700.0
freezer->bakery 500.0 r2 500.0
produce->dairy 102.5 r1 105.0
```

Constraints:

Similar to how you implemented the pipeline in Project 5, write your Map and Reduce code in Python3 using the Hadoop Streaming Interface with key-value pairs separated by a tab `(\t)` character. Read all input from stdin and write all output to stdout.

1. You must meaningfully use all outlined stages to receive credit. You may use the identity function (no transformation) as a Map stage.
2. All Map stages must use O(1) memory to receive credit.
3. All Map stages must produce multiple groups on the sample input to receive credit.

Hint: Python lists have a sort() member function. When the elements of the list are tuples, the list is sorted by the first element of the tuple, with ties broken by the second element of the tuple, and so on.

Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

```
#!/usr/bin/env python3
# map1.py
# Assume all needed packages are imported
```

```
# Input: [robot_id] [timestamp] [zone] [event]
# Output: [robot_id]\t[timestamp] [zone] [event]

for line in sys.stdin:
    robot_id, timestamp, zone, event = line.strip().split()

    # Emit robot_id as key, timestamp, zone, event as value
    print(f"{robot_id}\t{timestamp} {zone} {event}")
```

Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

```
#!/usr/bin/env python3
# reduce1.py
# Assume all needed packages are imported
```

```
# Input: [robot_id]\t[timestamp] [zone] [event]
# Output: [robot_id] [zone_transition_path] [duration]

def reduce_1_group(key, group):
    """Reduce one group for a robot."""
    records = []
    for line in group:
        robot_id, value = line.strip().split("\t")
        timestamp, zone, event = value.split()
        records.append((float(timestamp), zone, event))

    # Sort by timestamp
    records.sort()

    prev_zone = None
    prev_timestamp = None

    for ts, zone, event in records:
        if prev_zone is not None:
            # prev_zone being none means the robot is first seen
            if prev_zone != zone:
                print(f"{key} {prev_zone}->{zone} {ts-prev_timestamp}")
        prev_zone = zone
        prev_timestamp = ts
```

```
def keyfunc(line):
    """Return the key from a TAB-delimited key-value pair."""
    return line.partition("\t")[0]

def main():
    """Divide sorted lines into groups that share a key."""
    for key, group in itertools.groupby(sys.stdin, keyfunc):
        reduce_1_group(key, group)

if __name__ == "__main__":
    main()
```

Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

```
#!/usr/bin/env python3
# map2.py
# Assume all needed packages are imported
```

```
# Input: [robot_id] [zone_transition_path] [duration]
# Output: [zone_transition_path]\t[robot_id] [duration]

for line in sys.stdin:
    robot_id, zone_transition_path, duration  = line.strip().split()
    print(f"{zone_transition_path}\t{robot_id} {duration}")
```

Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

```
#!/usr/bin/env python3
# reduce2.py
# Assume all needed packages are imported
```

```
# Input: [zone_transition_path]\t[robot_id] [duration]
# Output: [robot_id] [avg_duration] [slowest_robot] [slowest_duration]

def reduce_2_group(key, group):
    """Reduce one group."""

    total_duration = 0
    num_times_taken = 0
    durations = []

    for line in group:
        parts = line.strip().split("\t")
        transition, temp = parts
        robot_id, duration = temp.split()
        duration = float(duration)
        durations.append((duration, robot_id))
        total_duration += duration
        num_times_taken +=1

    avg = total_duration/num_times_taken
    durations.sort() # sort by duration, with ties broken by robot_id

    greatest_duration = durations[-1][0]
    slowest_robot = durations[-1][1]
    print(key, avg, slowest_robot, greatest_duration)
```

```
def keyfunc(line):
    """Return the key from a TAB-delimited key-value pair."""
    return line.partition("\t")[0]

def main():
    """Divide sorted lines into groups that share a key."""
    for key, group in itertools.groupby(sys.stdin, keyfunc):
        reduce_2_group(key, group)

if __name__ == "__main__":
    main()
```

Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/